Projections

Projections

Introduction

Spring Data query methods usually return one or multiple instances of the aggregate root managed by the repository. However, it might sometimes be desirable to create projections based on certain attributes of those types. Spring Data allows modeling dedicated return types, to more selectively retrieve partial views of the managed aggregates.

Imagine a repository and aggregate root type such as the following example:

A sample aggregate and repository

JAVA

```java
class Person {

  @Id UUID id;
  String firstname, lastname;
  Address address;

  static class Address {
    String zipCode, city, street;
  }
}

interface PersonRepository extends Repository<Person, UUID> {

  Collection<Person> findByLastname(String lastname);
}
```

Now imagine that we want to retrieve the person's name attributes only. What means does Spring Data offer to achieve this? The rest of this chapter answers that question.

NOTE

Projection types are types residing outside the entity's type hierarchy. Superclasses and interfaces implemented by the entity are inside the type hierarchy hence returning a supertype (or implemented interface) returns an instance of the fully materialized entity.

Interface-based Projections

The easiest way to limit the result of the queries to only the name attributes is by declaring an interface that exposes accessor methods for the properties to be read, as shown in the following example:

A projection interface to retrieve a subset of attributes

JAVA

```
interface NamesOnly {

  String getFirstname();
  String getLastname();
}
```

The important bit here is that the properties defined here exactly match properties in the aggregate root. Doing so lets a query method be added as follows:

A repository using an interface based projection with a query method

JAVA

```
interface PersonRepository extends Repository<Person, UUID> {

  Collection<NamesOnly> findByLastname(String lastname);
}
```

The query execution engine creates proxy instances of that interface at runtime for each element returned and forwards calls to the exposed methods to the target object.

NOTE

Declaring a method in your `Repository` that overrides a base method (e.g. declared in `CrudRepository`, a store-specific repository interface, or the `Simple…Repository`) results in a call to the base method regardless of the declared return type. Make sure to use a compatible return type as base methods cannot be used for projections. Some store modules support `@Query` annotations to turn an overridden base method into a query method that then can be used to return projections.

Projections can be used recursively. If you want to include some of the `Address` information as well, create a projection interface for that and return that interface from the declaration of `getAddress()`, as shown in the following example:

A projection interface to retrieve a subset of attributes

```java
interface PersonSummary {

  String getFirstname();
  String getLastname();
  AddressSummary getAddress();

  interface AddressSummary {
    String getCity();
  }
}
```

On method invocation, the `address` property of the target instance is obtained and wrapped into a projecting proxy in turn.

Closed Projections

A projection interface whose accessor methods all match properties of the target aggregate is considered to be a closed projection. The following example (which we used earlier in this chapter, too) is a closed projection:

A closed projection

```java
interface NamesOnly {

  String getFirstname();
  String getLastname();
}
```

If you use a closed projection, Spring Data can optimize the query execution, because we know about all the attributes that are needed to back the projection proxy. For more details on that, see the module-specific part of the reference documentation.

Open Projections

Accessor methods in projection interfaces can also be used to compute new values by using the `@Value` annotation, as shown in the following example:

An Open Projection

```java
interface NamesOnly {

  @Value("#{target.firstname + ' ' + target.lastname}")
  String getFullName();
```

```
  …
}
```

The aggregate root backing the projection is available in the `target` variable. A projection interface using `@Value` is an open projection. Spring Data cannot apply query execution optimizations in this case, because the SpEL expression could use any attribute of the aggregate root.

The expressions used in `@Value` should not be too complex — you want to avoid programming in `String` variables. For very simple expressions, one option might be to resort to default methods (introduced in Java 8), as shown in the following example:

A projection interface using a default method for custom logic

JAVA

```
interface NamesOnly {

  String getFirstname();
  String getLastname();

  default String getFullName() {
    return getFirstname().concat(" ").concat(getLastname());
  }
}
```

This approach requires you to be able to implement logic purely based on the other accessor methods exposed on the projection interface. A second, more flexible, option is to implement the custom logic in a Spring bean and then invoke that from the SpEL expression, as shown in the following example:

Sample Person object

JAVA

```
@Component
class MyBean {

  String getFullName(Person person) {
    …
  }
}

interface NamesOnly {

  @Value("#{@myBean.getFullName(target)}")
  String getFullName();
  …
}
```

Notice how the SpEL expression refers to `myBean` and invokes the `getFullName(…)` method and forwards the projection target as a method parameter. Methods backed by SpEL expression evaluation can also use method parameters, which can then be referred to from the expression. The method parameters are available through an `Object` array named `args`. The following example shows how to get a method parameter from the `args` array:

Sample Person object

```java
interface NamesOnly {

  @Value("#{args[0] + ' ' + target.firstname + '!'}")
  String getSalutation(String prefix);
}
```

Again, for more complex expressions, you should use a Spring bean and let the expression invoke a method, as described earlier.

Nullable Wrappers

Getters in projection interfaces can make use of nullable wrappers for improved null-safety. Currently supported wrapper types are:

- `java.util.Optional`
- `com.google.common.base.Optional`
- `scala.Option`
- `io.vavr.control.Option`

A projection interface using nullable wrappers

```java
interface NamesOnly {

  Optional<String> getFirstname();
}
```

If the underlying projection value is not `null`, then values are returned using the present-representation of the wrapper type. In case the backing value is `null`, then the getter method returns the empty representation of the used wrapper type.

Class-based Projections (DTOs)

Another way of defining projections is by using value type DTOs (Data Transfer Objects) that hold properties for the fields that are supposed to be retrieved. These DTO types can be used in exactly the same

way projection interfaces are used, except that no proxying happens and no nested projections can be applied.

If the store optimizes the query execution by limiting the fields to be loaded, the fields to be loaded are determined from the parameter names of the constructor that is exposed.

The following example shows a projecting DTO:

A projecting DTO

```java
record NamesOnly(String firstname, String lastname) {
}
```

Java Records are ideal to define DTO types since they adhere to value semantics: All fields are `private final` and `equals(…)` / `hashCode()` / `toString()` methods are created automatically. Alternatively, you can use any class that defines the properties you want to project.

Dynamic Projections

So far, we have used the projection type as the return type or element type of a collection. However, you might want to select the type to be used at invocation time (which makes it dynamic). To apply dynamic projections, use a query method such as the one shown in the following example:

A repository using a dynamic projection parameter

```java
interface PersonRepository extends Repository<Person, UUID> {

  <T> Collection<T> findByLastname(String lastname, Class<T> type);
}
```

This way, the method can be used to obtain the aggregates as is or with a projection applied, as shown in the following example:

Using a repository with dynamic projections

```java
void someMethod(PersonRepository people) {

  Collection<Person> aggregates =
    people.findByLastname("Matthews", Person.class);

  Collection<NamesOnly> aggregates =
    people.findByLastname("Matthews", NamesOnly.class);
}
```

NOTE

Query parameters of type `Class` are inspected whether they qualify as dynamic projection parameter. If the actual return type of the query equals the generic parameter type of the `Class` parameter, then the matching `Class` parameter is not available for usage within the query or SpEL expressions. If you want to use a `Class` parameter as query argument then make sure to use a different generic parameter, for example `Class<?>`.

NOTE

When using Class-based projection, types must declare a single constructor so that Spring Data can determine their input properties. If your class defines more than one constructor, then you cannot use the type without further hints for DTO projections. In such a case annotate the desired constructor with `@PersistenceCreator` as outlined below so that Spring Data can determine which properties to select:

```
public class NamesOnly {

  private final String firstname;
  private final String lastname;

  protected NamesOnly() { }

  @PersistenceCreator
  public NamesOnly(String firstname, String lastname) {
      this.firstname = firstname;
      this.lastname = lastname;
  }

  // ...
}
```

Using Projections with JPA

You can use Projections with JPA in several ways. Depending on the technique and query type, you need to apply specific considerations.

Spring Data JPA uses generally `Tuple` queries to construct interface proxies for Interface-based Projections.

Derived queries

Query derivation supports both, class-based and interface projections by introspecting the returned type. Class-based projections use JPA's instantiation mechanism (constructor expressions) to create the projection instance.

Projections limit the selection to top-level properties of the target entity. Any nested properties resolving to joins select the entire nested property causing the full join to materialize.

String-based queries

Support for string-based queries covers both JPQL queries (`@Query`) and native queries (`@NativeQuery`).

JPQL Queries

JPA's mechanism to return Class-based projections using JPQL is **constructor expressions**. Therefore, your query must define a constructor expression such as `SELECT new com.example.NamesOnly(u.firstname, u.lastname) from User u` . (Note the usage of a FQDN for the DTO type!) This JPQL expression can be used in `@Query` annotations as well where you define any named queries. As a workaround you may use named queries with `ResultSetMapping` or the Hibernate-specific `ResultListTransformer` .

Spring Data JPA can aid with rewriting your query to a constructor expression if your query selects the primary entity or a list of select items.

DTO Projection JPQL Query Rewriting

JPQL queries allow selection of the root object, individual properties, and DTO objects through constructor expressions. Using a constructor expression can quickly add a lot of text to a query and make it difficult to read the actual query. Spring Data JPA can support you with your JPQL queries by introducing constructor expressions for your convenience.

Consider the following queries:

Example 1. Projection Queries

JAVA

```
interface UserRepository extends Repository<User, Long> {

  @Query("SELECT u FROM USER u WHERE u.lastname = :lastname")                    1
  List<UserDto> findByLastname(String lastname);

  @Query("SELECT u.firstname, u.lastname FROM USER u WHERE u.lastname = :lastname")  2
  List<UserDto> findMultipleColumnsByLastname(String lastname);
}

record UserDto(String firstname, String lastname){}
```

1 Selection of the top-level entity. This query gets rewritten to `SELECT new UserDto(u.firstname, u.lastname) FROM USER u WHERE u.lastname = :lastname` .

2 Multi-select of `firstname` and `lastname` properties. This query gets rewritten to `SELECT new UserDto(u.firstname, u.lastname) FROM USER u WHERE u.lastname = :lastname` .

WARNING

WARNING

JPQL constructor expressions must not contain aliases for selected columns and query rewriting will not remove them for you. While `SELECT u as user, count(u.roles) as roleCount FROM USER u …` is a valid query for interface-based projections that rely on column names from the returned `Tuple`, the same construct is invalid when requesting a DTO where it needs to be `SELECT u, count(u.roles) FROM USER u …`.
Some persistence providers may be lenient about this, others not.

Repository query methods that return a DTO projection type (a Java type outside the domain type hierarchy) are subject for query rewriting. If an `@Query`-annotated query already uses constructor expressions, then Spring Data backs off and doesn't apply DTO constructor expression rewriting.

Make sure that your DTO types provide an all-args constructor for the projection, otherwise the query will fail.

Native Queries

When using Class-based projections, their usage requires slightly more consideration depending on your :

- If properties of the result type map directly to the result (the order of columns and their types match the constructor arguments), then you can declare the query result type as the DTO type without further hints (or use the DTO class through dynamic projections).
- If the properties do not match or require transformation, use `@SqlResultSetMapping` through JPA's annotations map the result set to the DTO and provide the result mapping name through `@NativeQuery(resultSetMapping = "…")`.